

교과목 3 : 초거대언어모델(LLM)

단원 4 : 프롬프트 엔지니어링

DAY10. AI Agent 구현 실습 3

목 차

1. 통합 아키텍처		3
2. 프롬프트 엔지니어링 응용		29
3. 확장 방향		35

1. 통합 아키텍처

조각에서 제품으로

지금까지 **부품을 하나씩** 만들었습니다 — 도구·RAG·메모리·설계·운영.

이번 강은 이 부품들을 **설계대로 조립해 하나의 동작하는 제품**, "승승장구물 통합 CS 에이전트"를 완성합니다.

LLM 이 "답 못 했던" 질문을, 이제 우리 에이전트가 **실제 데이터를 조회해** 답합니다.

1. 여기까지 우리가 만든 부품들

지금까지 에이전트의 부품을 하나씩 익혔습니다.

도구 — 주문/재고/FAQ 조회, 함수 호출, 루프

RAG — 사내 정책 문서를 근거로 답변

메모리 — 대화 맥락·사용자 기억

멀티에이전트 — 역할 분담·협업

설계·배포 — 설정·로깅·예외처리·운영

각 부품은 **따로** 동작했습니다. 도구는 도구대로, RAG 는 RAG 대로, 메모리는 메모리대로.

이제 이걸 **하나로 합쳐** 진짜 상담원처럼 일하는 시스템을 만듭니다.

처음에 LLM 에게 이렇게 물었습니다.

"승승장구물 주문번호 0000123은 지금 배송 어디까지 왔나요?"

→ LLM: "죄송하지만 실시간 주문 정보는 확인할 수 없습니다" (답 못 함)

그때 우리는 "이걸 답하게 만드는 게 RAG 의 목표"라고 했습니다.

완성된 에이전트는:

고객: "내 주문 0000050은?"

상담원: "주문 0000050: 자기계발 다이어리 2개, 36,000원, 상태=배송완료입니다."

(get_order_status 도구가 orders.csv를 실제로 조회!)

말만 하던 LLM이, 실제로 일을 끝내는 에이전트가 되는 순간입니다.

우리는 **구성요소 필요성**을 판단했습니다. 그 결론을 그대로 따릅니다.

- ✔ 도구 4개 : `get_order_status`, `get_stock`, `search_faq`, `policy_search`
- ✔ RAG : 정책/멤버십 PDF (`policy_search` 안에서)
- ✔ 단기메모리 : `thread_id` 별 대화 세션
- ✗ 멀티에이전트: 도구 4개라 단일 `create_agent` 로 충분

왜 단일 에이전트? 이전 수업에서 판단했듯, 도구가 4개뿐이라 **단일 `create_agent`** 하나가 충분히 다릅니다. 멀티에이전트는 도구가 더 늘면 그때 도입하는 **확장 포인트**입니다. "필요 없는 건 안 넣는다"를 마지막까지 지킵니다.

. 핵심 통합 기법 — "RAG를 도구로 만든다"

여러 부품을 하나로 합치는 비결은 **"모든 능력을 도구로 통일"** 하는 것입니다.

```
주문 조회 → @tool get_order_status
재고 조회 → @tool get_stock
FAQ       → @tool search_faq
정책 검색 → create_retriever_tool 로 RAG를 '도구'로 감쌌 ← 핵심!
```

RAG(정책 검색)도 **하나의 도구**로 만들어, 다른 도구들과 **나란히** 에이전트에 넣습니다. 그러면 에이전트가 질문을 보고 스스로 고릅니다 — "이건 정책 검색, 저건 주문 조회". 능력을 도구로 통일하니 통합이 깔끔해지죠.

통합 아키텍처 한눈에

완성될 시스템의 **전체 그림**입니다.

사용자 질문이 들어오면 `create_agent` 가 **4 개 도구**(주문·재고·FAQ·정책 RAG) 중 알맞은 걸 골라 쓰고, `InMemorySaver` 가 대화를 기억하며, **설정·로깅**이 이 모두를 감쌉니다.

핵심은 **"RAG 도 도구로 만들어"** 모든 능력을 하나의 에이전트에 통일하는 것입니다.

1. 전체 아키텍처

위에서 아래로 흐릅니다.

1. `config.yaml` + `.env` (키) → 설정/로깅 로더(29강)
2. 사용자 질문 + `thread_id` → `create_agent` (system_prompt: 역할/규칙)
3. 에이전트가 아래 구성요소를 골라 사용:

구성요소	종류 / 소스
<code>get_order_status</code>	도구 → orders.csv
<code>get_stock</code>	도구 → inventory.csv
<code>search_faq</code>	도구 → faq.csv
<code>policy_search</code>	RAG → FAISS (정책 PDF)
<code>InMemorySaver</code>	thread_id별 단기기억

4. → 최종 답변

위에서 아래로 읽습니다:

설정 로드 → 에이전트 생성 → 도구/RAG 선택 실행 → 메모리 기록 → 답변

2. 각 요소와 담당 요구사항

요소	구현	담당 요구사항
주문 조회	<code>@tool get_order_status</code> (orders.csv)	R2
재고 조회	<code>@tool get_stock</code> (inventory.csv)	R3
FAQ	<code>@tool search_faq</code> (faq.csv)	R4
정책 RAG	<code>create_retriever_tool</code> (FAISS)	R1
단기 기억	<code>InMemorySaver</code> + <code>thread_id</code>	R5
설정·로깅	<code>config.yaml</code> + <code>logging</code>	운영

3. 핵심 통합 기법 — RAG를 도구로

이 아키텍처의 가장 중요한 발상입니다.

- [보통 생각] 도구는 도구, RAG는 RAG — 따로 처리
→ 에이전트 안에 'if 정책질문 then RAG else 도구' 분기 필요 (복잡)
- [통합 발상] RAG도 '도구'로 만들어 다른 도구와 나란히 넣음
→ 에이전트가 질문 보고 스스로 도구 선택 (분기 불필요)

`create_retriever_tool` 이 정책 검색 retriever를 하나의 도구로 감쌉니다. 그러면 `policy_search` 가 `get_order_status` 와 똑같은 도구가 되어, 에이전트가 알아서 고르죠.

```
tools = [  
    get_order_status,      # 도구  
    get_stock,            # 도구  
    search_faq,           # 도구  
    build_policy_tool(),  # RAG를 도구로 감싼 것 ← 나란히!  
]  
agent = create_agent(llm, tools=tools, ...)
```

에이전트는 "정책 질문이네 → `policy_search`", "주문 질문이네 → `get_order_status`"를 스스로 판단합니다. 능력을 도구로 통일한 덕입니다(함수 호출 + 통합 패턴).

4. 세 가지 능력이 한 에이전트에

완성된 에이전트는 한 대화 안에서 세 가지를 함께 씁니다.

- "환불 절차 알려줘" → RAG (정책 PDF 검색 → 근거 있는 답)
"내 주문 0000050은?" → 도구 (orders.csv 실시간 조회)
"아까 그 정책 다시" → 메모리 (이전 맥락 기억 → 환불로 해석)

질문 → 하나의 `create_agent` (질문 분석 → 알맞은 도구/RAG 선택 → 실행, + InMemorySaver가 대화 맥락 유지) → 답변

따로 배운 RAG·도구·메모리가 한 에이전트 안에서 협력합니다. 이게 30강 졸업작품의 핵심이고, 06번에서 직접 시연합니다.

이 에이전트는 진공 속에 있지 않습니다. 29강의 운영 계층이 둘러싸입니다.

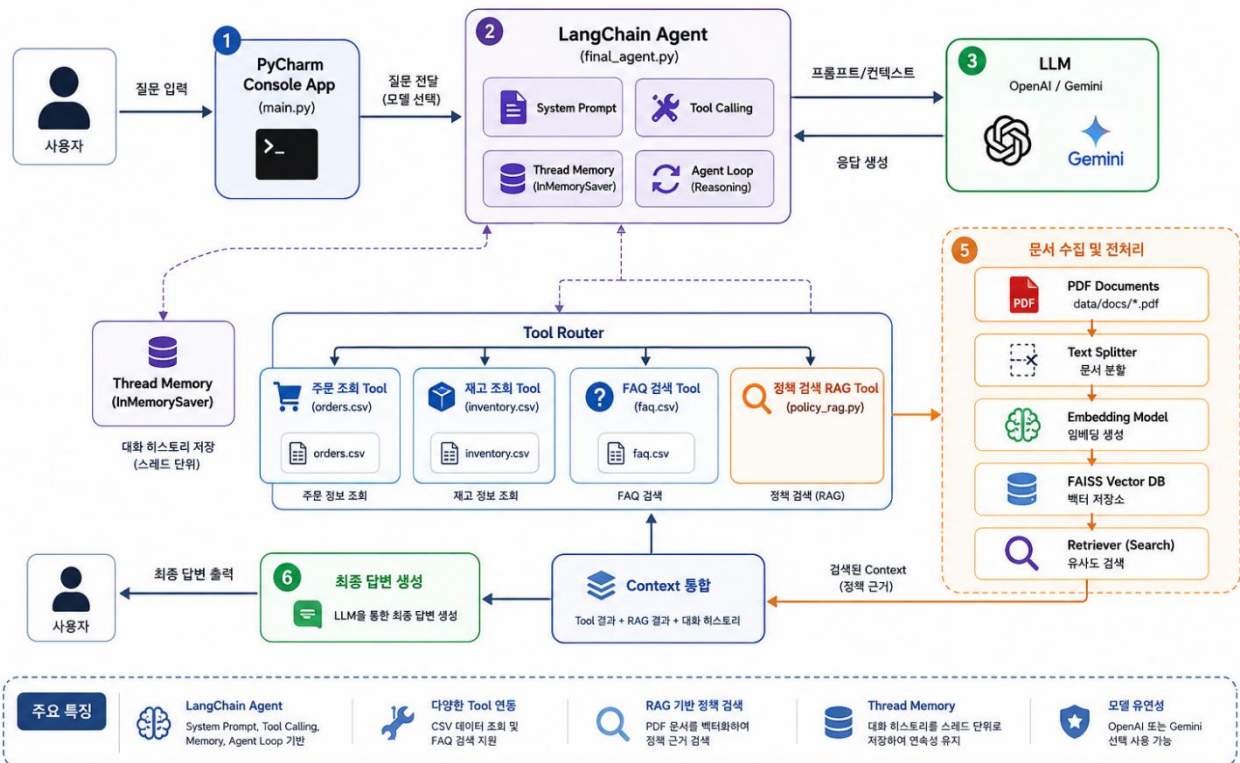
- 설정: config.yaml 에서 모델 · 온도 · RAG 파라미터(top_k 등) 로드
- 로깅: 인덱싱 · 도구 호출 · 에러를 logs/agent.log 에 기록
- 진입점: answer(message, thread_id) — 28강 설계의 단일 진입점
- (선택) FastAPI: 29강 /chat 에 answer()를 끼우면 HTTP 서비스

비즈니스 로직(에이전트)과 운영 계층(설정·로깅·HTTP)이 분리돼 있어, 본체는 그대로 두고 운영만 갈아끼울 수 있습니다.

핵심 정리

- 전체 흐름: 설정 로드 → create_agent → 도구/RAG 선택 → 메모리 기록 → 답변.
- 요구사항 R1~R5 가 각 요소로 빠짐없이 구현(주문·재고·FAQ·정책 RAG·메모리).
- 핵심 기법: RAG 를 create_retriever_tool 로 도구화 → 다른 도구와 나란히 → 에이전트가 스스로 선택.
- RAG·도구·메모리 세 능력이 한 에이전트에서 협력(06 번 시연).
- 설정·로깅·진입점이 운영 계층으로 감쌌(본체와 분리).

Final System Console Project - 시스템 구성도



따라하기 — 도구 3 개 (실데이터 조회)

목표

통합 에이전트의 첫 부품인 도구 3개를 만듭니다 — 주문 조회·재고 조회·FAQ 검색. 모두 승승장구물의 실제 CSV 데이터를 조회합니다. 실제 파일은 `code/ch30_final.py` 입니다. (28강 스켈레톤의 TODO를 채우는 과정입니다.)

0. 실습 준비

```
conda activate agentic

# 30강 통합에 필요한 라이브러리(전부 이전 강에서 설치한 것들)
# - langgraph, langchain : create_agent, 메모리
# - langchain-community : PDF 로더 · FAISS (04번 RAG에서)
# - faiss-cpu, pypdf : 벡터 검색 · PDF (04번)
# - pyyaml : config 로드 (29강)
pip install -U langgraph langchain langchain-google-genai langchain-community faiss-cpu pypdf pyyaml

python code/common.py      # GOOGLE_API_KEY: True 확인
```

- 데이터: `orders.csv`, `inventory.csv`, `faq.csv` (+ 04번에서 정책 PDF).

1. 시작부

`code/ch30_final.py` 를 만듭니다. 설정·로깅을 그대로 가져옵니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch30_final.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import get_chat, get_embeddings, DATA, DOCS
from ch29_deploy import load_config, setup_logging             # 29강 자산(설정 · 로깅) 재사용

import pandas as pd
from langchain_core.tools import tool
```



```
import pandas as pd
from langchain_core.tools import tool

cfg = load_config()          # 29강 설정 로드(모델 · 온도 · RAG 파라미터)
logger = setup_logging(cfg)  # 29강 구조적 로깅

orders = pd.read_csv(DATA / "orders.csv")      # 주문 데이터
inventory = pd.read_csv(DATA / "inventory.csv") # 재고 데이터
faq = pd.read_csv(DATA / "faq.csv")            # FAQ 데이터
```

- from ch29_deploy import load_config, setup_logging — 이전 수업에서 만든 **운영 자산을 재사용**. 설정·로깅을 다시 안 만듭니다.
- CSV 3 개를 **모듈 로드 시 한 번** 읽어 전역에 둡니다(매 호출마다 안 읽게).

2. 주문 조회 도구 (R2)

```
@tool
def get_order_status(order_id: str) -> str:
    """[R2] 주문번호(예: 0000050)로 주문 상태/상품/금액을 조회한다.

    @tool docstring 은 LLM이 읽는 도구 설명서다. LLM은 주문 관련 질문이 오면 이 도구를 고른다.
    """
    hit = orders[orders["order_id"] == order_id.strip().upper()]
    if hit.empty:
        return f"주문번호 {order_id} 를 찾을 수 없습니다."
    r = hit.iloc[0]
    return (f"주문 {r.order_id}: {r.product_name} {r.quantity}개, "
            f"{int(r.amount):,}원, 주문일 {r.order_date}, 상태={r.status}")
```

핵심

- **@tool** 데코레이터 — 함수를 에이전트가 쓸 수 있는 도구로 만듦(5강).
- 독스트링이 도구 설명서 — LLM이 이 설명을 읽고 "주문 질문엔 이 도구"를 고릅니다. 그래서 독스트링을 명확히 씁니다(6강).
- **order_id.strip().upper()** — 입력 정규화(공백 제거, 대문자). "o000050"도 "O000050"으로.
- 없으면 "찾을 수 없습니다" — 에러가 아니라 친절한 메시지 반환(07번 에러 처리의 기초).

3. 재고 조회 도구 (R3)

```
@tool
def get_stock(product_name: str) -> str:
    """[R3] 상품명(일부)으로 재고 수량을 조회한다."""
    hit = inventory[inventory["product_name"].str.contains(product_name, na=False)]
    if hit.empty:
        return f"'{product_name}' 상품 재고 정보를 찾을 수 없습니다."
    return "\n".join(f"{r.product_name}: 재고 {r.stock}개({r.warehouse}창고)"
                    for r in hit.head(3).itertuples())
```

핵심

- `str.contains(product_name)` — 상품명 일부로 검색. "이어버드"로 "무선 이어버드 프로"를 찾음.
- `na=False` — 결측값(NaN)을 False로 처리해 에러 방지.
- `head(3)` — 여러 개 매칭돼도 상위 3개만(너무 길어지지 않게).

4. FAQ 검색 도구 (R4)

```
@tool
def search_faq(keyword: str) -> str:
    """[R4] 키워드로 FAQ에서 관련 답변을 찾는다."""
    hit = faq[faq["question"].str.contains(keyword, na=False)]
    if hit.empty:
        hit = faq[faq["answer"].str.contains(keyword, na=False)] # 질문에 없으면 답변에서도
    if hit.empty:
        return "관련 FAQ를 찾지 못했습니다."
    return "\n".join(f"Q.{r.question}\nA.{r.answer}" for r in hit.head(2).itertuples())
```

핵심

- 2단계 검색 — 먼저 질문(`question`)에서 찾고, 없으면 답변(`answer`)에서도 검색. 적중률을 높임.
- 못 찾으면 "관련 FAQ를 찾지 못했습니다" — 역시 친절한 메시지.

5. 도구의 공통 설계 원칙

세 도구가 공유하는 좋은 설계 패턴입니다.

- ① 명확한 독스트링: [R2] 같은 요구사항 번호 + 무엇을 하는지 → LLM이 잘 고름
- ② 입력 정규화: `strip().upper()`, `str.contains` → 사용자 입력의 사소한 차이 흡수
- ③ 친절한 실패: 없으면 예러가 아니라 "찾을 수 없습니다" 문자열 반환
- ④ 적당한 길이: `head(2~3)`으로 결과를 제한 → 토큰 절약(29강 비용)

특히 ③ **친절한 실패**가 중요합니다. 도구가 예외를 던지면 에이전트가 멈추지만, 문자열로 "없음"을 반환하면 에이전트가 그걸 읽고 "주문을 찾을 수 없네요"라고 자연스럽게 안내합니다. 이게 07번 에러 처리의 1차 방어선입니다.

6. 도구 단위로 테스트

에이전트에 넣기 전에, 도구만 따로 호출해 확인할 수 있습니다.

```
# @tool 로 만든 함수는 .invoke({"인자명": 값}) 로 직접 호출
print(get_order_status.invoke({"order_id": "0000050"}))
# 주문 0000050: 자기계발 다이어리 2개, 36,000원, 주문일 2025-10-29, 상태=배송완료

print(get_stock.invoke({"product_name": "다이어리"}))
print(search_faq.invoke({"keyword": "무료배송"}))
```

도구를 단위로 먼저 검증하면, 나중에 에이전트가 이상하게 답할 때 "도구가 문제인지, LLM 판단이 문제인지" 구분할 수 있습니다.

핵심 정리

- 통합의 첫 부품: **도구 3개**(`get_order_status`-`get_stock`-`search_faq`) — 실제 CSV 조회.
- **load_config·setup_logging 재사용** — 설정·로깅을 다시 안 만들.
- 도구 설계: **명확한 독스트링 + 입력 정규화 + 친절한 실패 + 적당한 길이**.
- **친절한 실패**(없으면 문자열 "없음" 반환)가 07번 에러 처리의 1차 방어선.
- `.invoke({...})`로 **도구 단위 테스트** 가능.

따라하기 — 정책 RAG 를 도구로

목표

마지막 네 번째 도구 — 정책 RAG를 만듭니다. 정책/멤버십 PDF를 임베딩→FAISS→retriever로 만든 뒤, `create_retriever_tool` 로 하나의 도구로 감쌉니다. 이게 "RAG를 도구로" 통합 기법의 핵심입니다.

1. 왜 RAG를 "도구"로 만드나

03번에서 만든 주문·재고·FAQ는 모두 `@tool` 입니다. 정책 검색도 같은 도구로 만들면, 에이전트가 다른 도구들과 똑같이 다룰 수 있습니다.

[RAG를 따로 두면]

에이전트 안에 "정책 질문이면 RAG, 아니면 도구" 분기 코드 필요 (복잡)

[RAG를 도구로 만들면]

`policy_search` 도 `get_order_status` 와 같은 '도구'

→ 에이전트가 질문 보고 스스로 선택 (분기 불필요!)

`create_retriever_tool` 이 RAG 검색기를 도구로 감싸 줍니다. 그러면 정책 검색이 네 번째 도구가 되어 나란히 놓이죠.

2. RAG 파이프라인 복습 (13~16강)

정책 도구를 만들려면 RAG 5단계를 거칩니다.

- ① PDF 로드 : 정책 PDF들을 페이지 문서로 읽기
- ② 청크 분할 : 긴 문서를 검색 단위로 잘게 자르기
- ③ 임베딩 : 각 청크를 벡터로 변환
- ④ FAISS 색인 : 벡터들을 검색 가능한 인덱스로
- ⑤ retriever : 질문과 관련된 청크를 찾아주는 검색기
→ `create_retriever_tool` 로 도구화

이번엔 마지막에 도구로 감싸는 단계가 추가됩니다.

3. 정책 도구 빌더

```
def build_policy_tool():
    """정책/멤버십 PDF를 임베딩→FAISS→retriever→도구로 만든다."""
    from langchain_community.document_loaders import PyPDFLoader
    from langchain_text_splitters import RecursiveCharacterTextSplitter
    from langchain_community.vectorstores import FAISS
    from langchain_core.tools.retriever import create_retriever_tool

    logger.info("정책 문서 인덱싱 시작(RAG)")
    docs = []
    for name in ["환불교환정책.pdf", "멤버십정책.pdf"]:
        docs += PyPDFLoader(str(DOCS / name)).load() # ① PDF들을 페이지 문서로 로드
    splitter = RecursiveCharacterTextSplitter( # ② 긴 문서를 검색 단위로 분할
        chunk_size=cfg["rag"]["chunk_size"], chunk_overlap=cfg["rag"]["chunk_overlap"])
    chunks = splitter.split_documents(docs)
    vs = FAISS.from_documents(chunks, get_embeddings()) # ③④ 임베딩 → FAISS 색인
    retriever = vs.as_retriever(search_kwargs={"k": cfg["rag"]["top_k"]}) # ⑤ 상위 k개 검색기
    logger.info(f"정책 문서 인덱싱 완료(청크 {len(chunks)}개)")
    return create_retriever_tool( # 검색기를 '도구'로 래핑
        retriever, "policy_search",
        "환불/교환/멤버십 등 승승장구물 사내 정책 문서를 검색한다.") # ← LLM이 읽는 도구 설명
```

한 줄씩 보겠습니다.

4. 단계별 해설

① PDF 로드

```
for name in ["환불교환정책.pdf", "멤버십정책.pdf"]:
    docs += PyPDFLoader(str(DOCS / name)).load()
```

정책 PDF 2개를 읽어 페이지 단위 문서로 만듭니다. `+=`로 두 PDF의 문서를 한 리스트에 모읍니다.

② 청크 분할

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=cfg["rag"]["chunk_size"], chunk_overlap=cfg["rag"]["chunk_overlap"])
chunks = splitter.split_documents(docs)
```

긴 문서를 검색 단위(청크)로 자릅니다. 크기·겹침을 **config**에서 읽어, 코드 수정 없이 튜닝(29강).
`config.yaml`엔 `chunk_size: 500, chunk_overlap: 50`.

③④ 임베딩 + FAISS

```
vs = FAISS.from_documents(chunks, get_embeddings())
```

청크들을 벡터로 변환(임베딩)해 FAISS 인덱스를 구축합니다. `get_embeddings()`는 `common.py`의 헬퍼(14강).

⑤ retriever

```
retriever = vs.as_retriever(search_kwargs={"k": cfg["rag"]["top_k"]})
```

질문과 유사한 청크 상위 **k**개를 찾는 검색기. `top_k`도 config에서(과하면 비용 ↑, 29강).

5. 핵심 — create_retriever_tool

마지막 한 줄이 통합의 열쇠입니다.

```
return create_retriever_tool(
    retriever, "policy_search",
    "환불/교환/멤버십 등 승승장구몰 사내 정책 문서를 검색한다.")
```

`create_retriever_tool(검색기, 도구이름, 도구설명):`

- retriever : 방금 만든 정책 검색기
- "policy_search": 도구 이름 (에이전트가 부를 이름)
- "환불/교환..." : 도구 설명 = LLM이 읽고 "정책 질문엔 이 도구"를 판단하는 근거

이 한 줄로 RAG 검색기가 **@tool**과 똑같은 도구가 됩니다.

세 번째 인자(설명)가 특히 중요 — 03 번 도구들의 독스트링과 같은 역할로, LLM이 이 도구를 언제 쓸지 판단합니다(도구 description).

6. 왜 정책만 RAG인가

네 도구 중 정책만 RAG인 이유를 짚어봅시다(28강 판단 복습).

주문 · 재고 · FAQ → CSV의 '구조화 데이터' → 그냥 조회(@tool)면 됨
정책 → PDF의 '긴 비구조 문서' → 검색해 근거로 답해야(RAG)

주문은 orders[orders["order_id"]==...]로 정확히 조회됩니다. 하지만 정책은 긴 문장 문서라, "환불 며칠?"에 답하려면 **관련 부분을 검색**해야 하죠. 그래서 정책만 RAG 입니다. 데이터 성격에 맞는 도구를 고른 것(구성요소 판단).

핵심 정리

- 마지막 도구 **정책 RAG** — PDF 를 임베딩→FAISS→retriever 로 만든 뒤 도구로 감쌌.
- RAG 5 단계(로드→분할→임베딩→색인→검색)로, config 로 파라미터 관리.
- **create_retriever_tool**(검색기, 이름, 설명) — RAG 를 @tool 과 같은 도구로 만드는 열쇠.
- 도구 **설명**이 LLM 의 선택 근거 — 명확히 써야 정책 질문에 이 도구를 고름.
- **정책만 RAG** 인 이유: 긴 비구조 문서라 검색이 필요(주문·재고는 구조화 → 조회).

따라하기 — 통합 에이전트 (도구 + RAG + 메모리)

목표

03·04번에서 만든 도구 4개에 단기 메모리를 더해, 하나의 `create_agent` 로 통합합니다. 시스템 프롬프트로 역할을 정하고, `InMemorySaver` 로 대화를 기억하며, 28강 설계의 단일 진입점 `answer()` 를 완성합니다.

1. 시스템 프롬프트 — 에이전트의 역할 규칙

에이전트가 어떻게 행동할지 정하는 규칙입니다.

```
SYSTEM_PROMPT = (
    "너는 승승장구몰의 통합 CS 상담원이다. 친절한 한국어로 답하라.\n"
    "- 주문 상태는 get_order_status, 재고는 get_stock, 자주 묻는 질문은 search_faq,\n"
    "  환불/교환/멤버십 정책은 policy_search 도구를 사용하라.\n"
    "- 정책 질문은 반드시 policy_search 결과(문서 근거)로만 답하고, 모르면 모른다고 하라.\n"
    "- 이전 대화 맥락(예: '아까 그 정책')을 기억해 이어서 답하라."
)
```

핵심 지시 3가지

- ① 어떤 질문에 어떤 도구를 쓸지 안내 (도구 선택 힌트)
- ② "정책은 문서 근거로만, 모르면 모른다" → 환각 방지 (16강 정신)
- ③ "이전 맥락을 기억해 이어 답하라" → 메모리 활용 유도

특히 ②가 중요합니다. "지어내지 말고 문서 근거로만"이 환각을 막는 안전장치죠.

2. 에이전트 빌더 — 통합의 핵심

도구 4개 + 메모리를 `create_agent` 로 결합합니다.

```
_agent = None # 모듈 캐시 (PDF 인덱싱이 비싸서 한 번만 빌드)

def build_agent():
    """설정+도구+RAG+메모리를 결합한 단일 에이전트를 만든다(30강 통합의 핵심)."""
    from langchain.agents import create_agent
    from langgraph.checkpoint.memory import InMemorySaver

    llm = get_chat(provider=cfg["llm"]["provider"], temperature=cfg["llm"]["temperature"])
    # 도구 4종 결합: 주문/재고/FAQ 조회 + 정책 RAG 검색
    tools = [get_order_status, get_stock, search_faq, build_policy_tool()]
    agent = create_agent(
        llm, tools=tools, system_prompt=SYSTEM_PROMPT,
        checkpoint=InMemorySaver() # InMemorySaver = 단기 메모리(thread_id 별 대화 기록)
    )
    logger.info(f"에이전트 구성 완료(도구 {len(tools)}개)")
    return agent
```

- **tools = [...]** — 03 번 도구 3 개 + 04 번 정책 RAG 도구. **RAG** 가 다른 도구와 나란히 들어갑니다(통합 기법).

3. 왜 모듈 캐시(_agent)인가

```
_agent = None # 모듈 캐시

def answer(message, thread_id="demo"):
    global _agent
    if _agent is None:          # 최초 호출에만 빌드
        _agent = build_agent()
    ...
```

`build_agent()` 는 PDF 인덱싱(임베딩→FAISS)을 합니다. 이건 시간·비용이 큰 작업이죠.

[캐시 없음] `answer()` 부를 때마다 `build_agent()` → 매번 PDF 재인덱싱 (낭비!)
[캐시 있음] 최초 1회만 빌드 → `_agent`에 저장 → 이후엔 재사용 (빠름)

전역 `_agent` 에 한 번 만든 에이전트를 저장해두고 재사용합니다. 서버라면 매 요청마다 재인덱싱하는 참사를 막죠(29강 비용 관리, 07번 FastAPI 1회 초기화와 같은 정신).

4. 단일 진입점 answer (28강 설계)

외부에 노출되는 함수입니다.

```
def answer(message: str, thread_id: str = "demo") -> str:
    """외부 단일 진입점(28강 설계). 같은 thread_id = 같은 대화로 묶인다."""
    global _agent
    if _agent is None:
        _agent = build_agent()
    # config 의 thread_id 로 대화 세션을 구분 → 같은 id면 이전 대화를 이어 기억
    config = {"configurable": {"thread_id": thread_id}}
    result = _agent.invoke({"messages": [{"role": "user", "content": message}]}, config)
    return result["messages"][-1].content
```

핵심

- `answer(message, thread_id)` — 28강에서 설계한 단일 진입점 그대로. 외부(CLI·FastAPI)는 이것만 부르면 됩니다.

5. 통합 구조 한눈에

지금까지 만든 걸 조립하면:

`answer(message, thread_id)` 는 두 가지를 합니다.

- (최초 1회) `build_agent()`
- `llm` (config의 모델·온도)
- `tools = [주문, 재고, FAQ, 정책RAG]` ← 03·04번
- `checkpointer = InMemorySaver()` ← 메모리(19강)
- `_agent.invoke(메시지, thread_id)` → 에이전트가 도구/RAG 선택 실행 + 대화 기억 → 답변 반환

28강 설계도(`AgentSpec`)의 **TODO**가 모두 채워졌습니다. `use_rag=True` (정책 도구),
`use_memory=True` (InMemorySaver), `use_multi_agent=False` (단일) — 설계 그대로 구현됐죠.

6. 설계 → 구현의 완성

28강 스케레톤과 비교해 봅시다.

[28강 스케레톤]		[30강 구현]
<pre>def build_agent(): # TODO: tools, checkpointer raise NotImplementedError</pre>	→	<pre>def build_agent(): tools = [...4개...] create_agent(..., checkpointer=InMemorySaver())</pre>
<pre>def answer(message, thread_id): # TODO: agent.invoke(...) raise NotImplementedError</pre>	→	<pre>def answer(message, thread_id): _agent.invoke({"messages":...}, config) return result["messages"][-1].content</pre>

시그니처만 삭제했던 함수들의 **TODO** 를 채웠습니다. 설계가 흔들림 없이 구현으로 이어진 거죠 — 이게 "설계 먼저"의 결실입니다.

핵심 정리

- 시스템 프롬프트로 역할·도구 선택·환각 방지·메모리 활용을 지시.
- `build_agent()`: 도구 4개(RAG 포함) + InMemorySaver(메모리)를 `create_agent` 로 결합 — 통합의 핵심.
- 모듈 캐시(`_agent`): PDF 인덱싱이 비싸므로 최초 1 회만 빌드 후 재사용.
- `answer(message, thread_id)`: 설계의 단일 진입점, `thread_id` 로 세션 구분.

따라하기 — 멀티턴 시나리오 데모

목표

완성된 통합 에이전트를 멀티턴 대화로 시연합니다. 한 세션 안에서 **RAG·도구·메모리** 세 능력이 함께 작동하는 걸 직접 확인합니다. 이게 30강 졸업작품의 하이라이트입니다.

1. 데모 시나리오 — 세 능력을 한 번에

한 고객이 세 가지 질문을 순서대로 던집니다. 각 질문이 다른 능력을 씁니다.

```
def main():
    tid = "user-1001" # 한 사람의 대화 세션(같은 id로 호출해야 맥락이 이어짐)
    turns = [
        "환불 절차 알려줘",          # 1) RAG: 정책 문서 근거로 답
        "내 주문 0000050은?",        # 2) 도구: 주문 조회
        "아까 그 정책 다시 알려줘",  # 3) 메모리: '아까 그 정책'=환불을 기억해 이어 답
    ]
    for i, q in enumerate(turns, 1):
        print("=" * 60)
        print(f"[턴 {i}] 고객:", q)
        print("상담원:", answer(q, thread_id=tid)) # 같은 tid → 같은 대화 세션

if __name__ == "__main__":
    main()
```

핵심: 세 질문 모두 같은 `thread_id="user-1001"` 로 호출합니다. 그래야 한 대화로 묶여 맥락이 이어지죠.

2. 실행 결과

```
=====
[턴 1] 고객: 환불 절차 알려줘
상담원: 환불은 상품 수령 후 7일 이내 신청 가능하며, 마이페이지 > 주문내역에서 ...
        (policy_search 도구가 환불교환정책.pdf 를 검색해 근거로 답함)
=====
```

[턴 2] 고객: 내 주문 0000050은?

상담원: 주문 0000050: 자기계발 다이어리 2개, 36,000원, 주문일 2025-10-29, 상태=배송완료 입니다.
(get_order_status 도구가 orders.csv 조회)

[턴 3] 고객: 아까 그 정책 다시 알려줘

상담원: 앞서 안내드린 환불 절차를 다시 정리하면, 수령 후 7일 이내 ...
(InMemorySaver 가 1턴의 맥락을 기억 → '아까 그 정책'을 환불로 해석)

세 턴이 각각 다른 능력을 썼습니다. 하나하나 뜯어봅시다.

3. 턴 1 — RAG (정책 문서 근거)

고객: "환불 절차 알려줘"

- 에이전트가 'policy_search' 도구 선택
- 환불교환정책.pdf 에서 관련 청크 검색
- 그 근거로 답변 생성

환각 없이 실제 정책 문서를 근거로 답합니다. LLM이 지어낸 게 아니라, FAISS가 찾은 문서 내용이죠 (13~18강 RAG). 시스템 프롬프트의 "정책은 문서 근거로만"이 작동한 결과입니다.

4. 턴 2 — 도구 (실시간 주문 조회)

고객: "내 주문 0000050은?"

- 에이전트가 'get_order_status' 도구 선택
- orders.csv 에서 0000050 행 조회
- 실제 주문 정보로 답변

1강에서 LLM이 못 답했던 바로 그 질문입니다! 이제 에이전트가 `orders.csv` 를 실제로 조회해 "다이어리 2개, 36,000원, 배송완료"라고 정확히 답합니다. 도구가 손발이 된 거죠(5강 함수 호출).

5. 턴 3 — 메모리 (맥락 기억)

고객: "아까 그 정책 다시 알려줘"

→ "아까 그 정책"이 뭐지? → InMemorySaver가 1턴 대화를 기억

→ "아, 환불 절차구나" → 이어서 답변

가장 인상적인 부분입니다. "아까 그 정책"이라는 모호한 지시를, 에이전트가 1턴의 "환불 절차"를 기억해 해석합니다. 사람 상담원처럼 맥락을 이어가죠(19강 메모리).

6. 세 능력의 협력 — 핵심 관찰

하나의 에이전트, 하나의 세션(`thread_id="user-1001"`) 안에서 세 능력이 함께 작동합니다.

- 턴 1: **RAG** — 정책 문서 검색 (환각 방지)
- 턴 2: **도구** — 주문 DB 실시간 조회
- 턴 3: **메모리** — 이전 맥락("아까 그 정책") 해석

따로 배운 RAG·도구·메모리가 한 대화 안에서 협력합니다. 이게 우리가 30강에 걸쳐 쌓아온 결과입니다 — "말만 하던 LLM"이 "실제로 일하는 에이전트" 가 됐죠.

직접 실험: 턴 3을 다른 `thread_id` 로 호출해 보세요. 예: `answer("아까 그 정책 다시", thread_id="other")`. 그러면 에이전트가 "어떤 정책을 말씀하시나요?"라고 되물습니다 — 다른 세션이라 1·2턴을 기억 못 하니까요. 이게 **메모리 격리**의 증거입니다(`thread_id`별 분리, 19강).

핵심 정리

- 멀티턴 시나리오로 **RAG·도구·메모리**가 한 세션에서 협력을 시연.
- 턴 1 **RAG**(정책 근거), 턴 2 **도구**(주문 조회 = 1 강의 그 질문!), 턴 3 **메모리**("아까 그 정책").
- 같은 `thread_id` 라서 맥락이 이어짐 — 다른 id 면 "어떤 정책요?"(메모리 격리).
- 29 강 /chat 에 `answer()`를 끼우면 그대로 **HTTP 서비스**(전송 계층 분리의 결실).
- 말만 하던 LLM 이 실제로 일하는 에이전트가 된 시연.

에러 케이스 다뤄보기 (주문 없음·오타)

06 번은 정상 시나리오였습니다. 이번엔 일부러 잘못된 입력(없는 주문번호, 상품명 오타, 정책에 없는 질문, 맥락 없는 지시어)을 던집니다. 잘 만든 에이전트는 거짓을 지어내지 않고(환각 방지), 한계를 정직하게 말해야 합니다. 그걸 검증합니다.

1. 왜 에러 케이스를 일부러 테스트하나

정상 입력만 잘 되는 건 절반의 완성입니다. 실제 고객은 오타·없는 주문·엉뚱한 질문을 끊임없이 던집니다.

[나쁜 에이전트] 없는 주문 → "배송 중입니다" (그럴듯하게 지어냄 = 환각! 위험)
[좋은 에이전트] 없는 주문 → "해당 주문을 찾을 수 없습니다" (정직)

가장 위험한 건 환각입니다. 없는 주문에 가짜 운송장을 만들어내면 고객이 속조(1강의 그 위험). "모르면 모른다" 고 말하는 에이전트가 신뢰할 수 있는 에이전트입니다.

2. 에러 시나리오 4가지

각 시나리오와 기대 동작을 정의합니다.

```
SCENARIOS = [  
    ("주문 09999999 상태 알려줘",  
     "기대: 존재하지 않는 주문 → '찾을 수 없음' 안내(임의 상태 지어내기 금지)"),  
    ("'로봇청소기' 재고 있어?",  
     "기대: 오타/미존재 상품 → 재고 정보 없음 안내(수량 환각 금지)"),  
    ("승승장구물 우주배송 정책이 어떻게 돼?",  
     "기대: 정책 문서에 근거 없음 → 모른다/확인 불가 안내(없는 정책 창작 금지)"),  
    ("아까 그거 다시 알려줘",  
     "기대: 가리킬 직전 맥락 없음 → 무엇인지 명확화 요청(아무거나 추측 금지)"),  
]
```

- ① 없는 주문번호 (09999999) → "찾을 수 없음"
- ② 상품명 오타 ('로봇청소기') → "재고 정보 없음"
- ③ 정책에 없는 질문 (우주배송) → "모른다/확인 불가"
- ④ 맥락 없는 지시어 (아까 그거) → "무엇인지 되물기"

각각 기대 동작을 미리 적어, 실제 출력과 대조합니다.

3. 1차 방어선 — 도구 단위 점검

LLM에 넣기 전에, 도구가 에러 입력을 어떻게 처리하는지 직접 봅니다.

```
def check_tools_directly() -> None:
    """LLM 없이 도구 단위 동작부터 확인한다(에러 처리의 1차 방어선)."""
    print(" 없는 주문:", get_order_status.invoke({"order_id": "0999999"}))
    print(" 오타 상품:", get_stock.invoke({"product_name": "로봇청소기"}))
    print(" 엉뚱 FAQ:", search_faq.invoke({"keyword": "우주배송"}))
```

예상 출력

```
없는 주문: 주문번호 0999999 를 찾을 수 없습니다.
오타 상품: '로봇청소기' 상품 재고 정보를 찾을 수 없습니다.
엉뚱 FAQ: 관련 FAQ를 찾지 못했습니다.
```

03번에서 도구마다 "친절한 실패 메시지"를 반환하게 만든 게 여기서 빛을 발합니다. 도구가 예외를 던지지 않고 "없음"을 문자열로 반환하니, 에이전트가 그걸 읽고 자연스럽게 안내할 수 있죠. 이게 에러 처리의 1차 방어선입니다.

4. 2차 방어선 — 에이전트의 정직함

이제 같은 에러 입력을 에이전트에 던집니다.

```
if __name__ == "__main__":
    check_tools_directly()

    tid = "edge-test-1"
    for i, (q, expect) in enumerate(SCENARIOS, 1):
        print(f"[턴 {i}] 고객:", q)
        print(f"  ↳ {expect}")
        print("상담원:", answer(q, thread_id=tid))
```

예상 출력

```
[턴 1] 고객: 주문 0999999 상태 알려줘
  ↳ 기대: 존재하지 않는 주문 → '찾을 수 없음' 안내
상담원: 주문번호 0999999는 확인되지 않습니다. 번호를 다시 확인해 주시겠어요?
```

[턴 2] 고객: '로봇청소기' 재고 있어?

↳ 기대: 오타/미존재 상품 → 재고 정보 없음 안내

상담원: '로봇청소기' 상품의 재고 정보를 찾을 수 없습니다. 정확한 상품명을 알려주시면 ...

[턴 3] 고객: 승승장구물 우주배송 정책이 어떻게 돼?

↳ 기대: 정책 문서에 근거 없음 → 모른다 안내

상담원: 우주배송 정책은 확인되지 않습니다. 저희가 안내 가능한 정책은 환불/교환/멤버십입니다.

[턴 4] 고객: 아까 그거 다시 알려줘

↳ 기대: 직전 맥락 없음 → 명확화 요청

상담원: '아까 그거'가 무엇인지 구체적으로 말씀해 주시겠어요?

5. 관찰 — 환각 없이 정직하게

각 응답의 핵심:

턴 1: 가짜 배송 상태를 만들지 않음 → "확인되지 않습니다" (환각 방지 ✓)

턴 2: 임의 재고 수량을 지어내지 않음 → "찾을 수 없습니다" (환각 방지 ✓)

턴 3: 없는 정책을 창작하지 않음 → "확인되지 않습니다" + 가능한 정책 안내 (환각 방지 ✓)

턴 4: 아무거나 추측하지 않음 → 무엇인지 되물음 (명확화 ✓)

좋은 에이전트의 핵심 덕목은 "모를 때 모른다고 하는 것"입니다. 시스템 프롬프트의 "모르면 모른다고 하라"(05번)와 도구의 "친절한 실패 메시지"(03번)가 함께 작동한 결과죠.

6. 두 겹의 안전망

에러 처리가 두 단계로 작동했습니다.

[1차: 도구 레벨] 없는 데이터 → "찾을 수 없습니다" 문자열 반환 (예외 안 던짐)

↓

[2차: 에이전트 레벨] 그 메시지를 읽고 → 고객에게 정중히 안내 + 대안 제시

도구가 **튼튼하게 실패**(03 번 친절한 메시지)하고, 에이전트가 **정직하게 전달**(05 번 환각 방지 프롬프트)합니다. 이 두 겹이 신뢰할 수 있는 서비스를 만듭니다.

실무 보강: 비가역 행동(환불 실행 등)엔 **human-in-the-loop**(사람 승인)을 더합니다. "환불해줘" → 바로 실행하지 말고 "정말 환불할까요?" 확인. 위험한 행동일수록 안전장치를 두는 거죠

핵심 정리

- 정상 시나리오(06 번)에 이어, **잘못된 입력**(없는 주문·오타·없는 정책·맥락 없는 지시)을 테스트.
- 가장 위험한 건 **환각** — 없는 정보를 지어내면 안 됨. "**모르면 모른다**" 가 핵심 덕목.
- **1차 방어선(도구)**: 친절한 실패 메시지 반환(예외 안 던짐, 03 번).
- **2차 방어선(에이전트)**: 그 메시지를 읽고 정직하게 안내(환각 방지 프롬프트, 05 번).
- 두 겹의 안전망이 **신뢰할 수 있는 서비스**를 만듦.
- 비가역 행동엔 **human-in-the-loop** 추가(실무 보강).

발표와 시연 가이드

완성한 에이전트를 **남에게 보여줄** 차례입니다.

작업을 **3분 데모**로 압축하는 시나리오와, 발표에서 강조할 핵심 메시지를 정리합니다.

포트폴리오·면접·팀 공유에 그대로 쓸 수 있습니다.

1. 왜 시연이 중요한가

아무리 잘 만들어도, 보여주지 못하면 가치가 전달되지 않습니다.

- 포트폴리오: "도구·RAG·메모리를 통합한 에이전트를 만들었다"를 3분에 증명
- 면접: 기술 나열이 아니라 '동작하는 결과물'로 설득
- 팀 공유: 동료가 5분 안에 "이게 뭘 하는지" 이해하게

좋은 시연은 "**문제 → 해결**"의 드라마를 보여줍니다. 그냥 "RAG 됩니다"가 아니라, "LLM이 못 하던 걸 이렇게 해냅니다"로요.

2. 3분 데모 시나리오 (추천)

가장 효과적인 5단계 흐름입니다.

- ① 문제 제기 (30초)
"LLM에게 '내 주문 0000050?'을 물으면 못 답합니다" (1강 회상)
→ 빈 두뇌의 한계를 먼저 보여줌

② 턴 1 — RAG (40초)

"환불 절차 알려줘" → 정책 문서 근거로 답변
→ "지어낸 게 아니라 PDF에서 찾은 겁니다" 강조

③ 턴 2 — 도구 (40초)

"내 주문 0000050은?" → 실제 orders.csv 조회해 답변
→ "아까 LLM이 못 한 걸, 이제 합니다!" (문제 해결의 순간)

④ 턴 3 — 메모리 (40초)

"아까 그 정책 다시" → 1턴 맥락 기억해 이어 답변
→ "사람 상담원처럼 맥락을 기억합니다" 강조

⑤ 운영성 (30초)

logs/agent.log 를 열어 로깅 · 재시도가 남는 것을 보여줌 (29강)
→ "데모가 아니라 운영 가능한 시스템입니다"

①에서 문제(LLM의 한계) 를 보여주고, ③에서 그 문제를 해결하는 순간이 클라이맥스입니다.

3. 발표 핵심 메시지

심사위원·면접관에게 각인시킬 한 문장들입니다.

핵심 1: "하나의 에이전트가 도구 · RAG · 메모리를 스스로 골라 씁니다"
→ 기술을 따로 나열하지 말고 '통합'을 강조

핵심 2: "설계(28강) → 운영화(29강) → 통합(30강)의 엔지니어링 흐름"
→ 코드만이 아니라 '계대로 만드는 과정'을 보여줌

핵심 3: "모르면 모른다고 합니다 (환각 방지)"
→ 신뢰성을 강조 (07번 예러 케이스 시연하면 더 강력)

기술 자랑이 아니라 "문제를 어떻게 풀었나" 의 이야기로 풀어가세요.

4. 시연 시 흔한 실수 피하기

- ❌ 인덱싱을 시연 중에 실행 → PDF 임베딩에 시간 걸려 어색한 침묵
 - ✅ 미리 한 번 실행해 _agent 캐시 워밍업(또는 인덱스 저장)
- ❌ 네트워크 의존 시연 → 현장 와이파이 불안정으로 LLM 호출 실패
 - ✅ 사전 리허설 + 폴백 메시지 확인(29강), 캐싱으로 대비(29강 08번)
- ❌ 너무 많은 기능 나열 → 3분에 다 보여주려다 아무것도 안 남음
 - ✅ RAG · 도구 · 메모리 '세 가지'만 또렷하게
- ❌ 코드만 스크롤 → 청중은 결과를 보고 싶어함
 - ✅ 실행 결과(대화)를 먼저, 코드는 핵심만

5. 시연 체크리스트

발표 전 점검표입니다.

- [] .env 에 GOOGLE_API_KEY 설정됨 (python code/common.py 확인)
- [] 정책 PDF 인덱싱 미리 1회 실행 (캐시 워밍업)
- [] 3개 톤 시나리오 리허설 완료 (RAG → 도구 → 메모리)
- [] logs/agent.log 에 로그가 쌓이는지 확인 (운영성 시연용)
- [] 예러 케이스(07번) 1개 준비 (환각 방지 강조용)
- [] 다른 thread_id로 '메모리 격리' 보여줄 준비 (선택)
- [] 백업 계획: 네트워크 실패 시 캐시된 결과나 녹화 영상

핵심 정리

- 잘 만들어도 보여주지 못하면 가치가 전달 안 됨 — 시연은 필수.
- 3분 데모: 문제 제기(1강 회상) → RAG → 도구 → 메모리 → 운영성(로그).
- 핵심 메시지: "하나의 에이전트가 도구·RAG·메모리를 스스로 통합" + 설계→운영→통합 흐름.
- 흔한 실수: 시연 중 인덱싱·네트워크 의존·기능 과다 나열 → 사전 리허설로 대비.
- 체크리스트로 발표 전 점검(키·캐시 워밍업·리허설·로그·백업).

6. 데모 멘트 예시

실제로 말할 멘트 흐름입니다.

"안녕하세요. LLM은 똑똑하지만, '내 주문 어디까지 왔어?' 같은 질문엔 답을 못 합니다. 실시간 데이터에 손이 안 닿으니까요. (문제 제기)

그래서 저는 도구 · RAG · 메모리를 통합한 CS 에이전트를 만들었습니다.

먼저 '환불 절차 알려줘' — 보세요, 정책 문서를 검색해 답합니다. 지어낸 게 아니에요.
다음 '내 주문 0000050은?' — 실제 주문 DB를 조회해 정확히 답합니다. 아까 LLM이 못 하던 거죠!
마지막 '아까 그 정책 다시' — 보세요, 앞 대화를 기억해 환불 절차를 다시 안내합니다.

하나의 에이전트가 세 능력을 스스로 골라 썼습니다. 그리고 이 모든 호출은 로그로 남고, 실패하면 재시도합니다 — 운영 가능한 시스템입니다. 감사합니다."

실습문제

문제 1 — 교환신청 도구 추가

요구사항

`code/ch30_final.py`를 복사해 `code/ch30_ex_exchange.py`를 만드세요.

- `@tool`로 `request_exchange(order_id: str, reason: str) -> str`를 정의해, 교환 접수번호(예: EX- + 난수)를 만들어 반환한다.
- `build_agent`의 `tools`에 추가한다.
- `SYSTEM_PROMPT`에 교환 요청 시 이 도구를 쓰라고 명시한다.
- 기대 결과: "주문 0000050 불량이라 교환하고 싶어" 입력 시 에이전트가 `request_exchange`를 호출하고 접수번호가 답변에 포함된다.

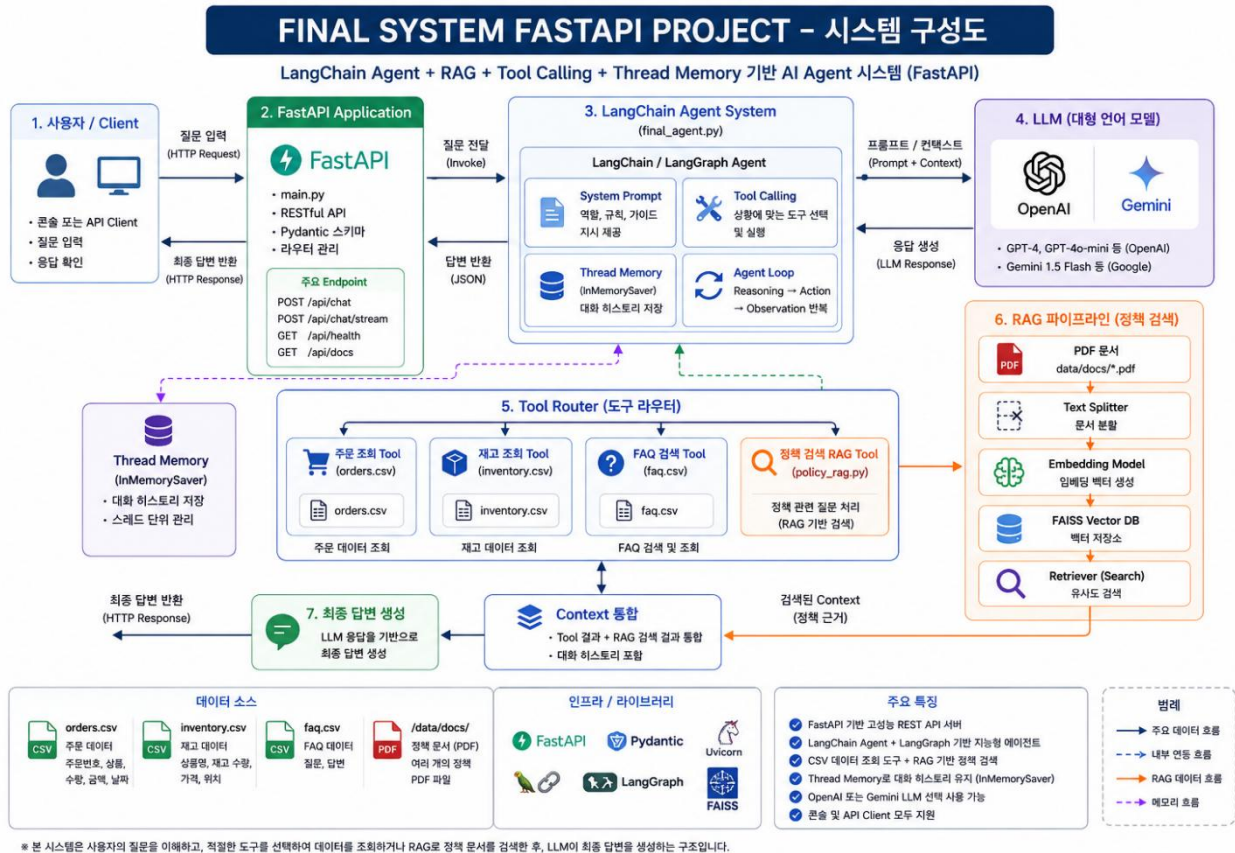
문제 2 — 메모리 격리 검증 스크립트

요구사항

`code/ch30_ex_memory.py`를 작성하세요.

- `answer()`를 사용해:
- ① `thread_id="A"`로 "내 이름은 오지표야" → "내 이름이 뭐였지?"를 연속 호출 → 두 번째 답에 "오지표"가 들어가는지(메모리 유지).
- ② `thread_id="B"`로 "내 이름이 뭐였지?"를 물었을 때는 기억하지 못하는지(스레드 격리).
- 검증: 콘솔에 "메모리 유지: PASS/FAIL", "스레드 격리: PASS/FAIL" 두 줄 출력.

2. 프롬프트 엔지니어링 응용



Fastapi 기반 final project 시스템 구성도입니다.

FastAPI 기반 ReAct + RAG + A2A + LangGraph + MCP 통합 AI Agent 프로젝트

본 프로젝트는 FastAPI 를 기반으로 구축된 통합 AI Agent 시스템으로, 최신 생성형 AI 기술과 에이전트 아키텍처를 하나의 플랫폼으로 통합하여 다양한 비즈니스 질의를 효율적으로 처리할 수 있도록 설계되었다.

프로젝트는 REST API 와 웹 기반 사용자 인터페이스를 제공하며, OpenAI 와 Google Gemini 를 모두 지원하여 사용 환경에 따라 원하는 대형 언어 모델을 선택하여 사용할 수 있다.

시스템의 핵심은 LangChain 과 LangGraph 를 활용한 AI Agent 이며, 사용자의 질문을 분석하여 적절한 처리 방식을 선택한다. 일반적인 질문은 LLM 이 직접 처리하고, 데이터 조회가 필요한 경우에는 다양한 Tool 을 호출하며, 정책과 같은 문서 기반 질문은 RAG(Retrieval-Augmented Generation)를 통해 근거를 검색한 후 답변을 생성한다.

또한 복합적인 질문은 ReAct(Reasoning + Acting) 패턴을 적용하여 질문 분석, 도구 선택, 실행 결과 관찰, 추가 도구 호출 및 최종 답변 생성 과정을 반복 수행함으로써 보다 정확한 응답을 제공한다.

프로젝트는 A2A(Agent-to-Agent) 구조를 적용하여 주문 조회, 재고 조회, FAQ 검색, 정책 검색, 교환 신청 등의 기능을 각각 독립적인 전문 에이전트로 구성하였다.

각 에이전트는 자신의 역할과 기능을 Agent Card 형태로 제공하며, 필요한 경우 다른 에이전트와 협력하여 하나의 질문을 처리한다. 이러한 구조는 기능의 독립성과 확장성을 높여 새로운 업무 기능을 손쉽게 추가할 수 있도록 지원한다.

정책 검색 기능은 RAG 기반으로 구현되어 있으며, PDF 형태의 정책 문서를 자동으로 수집하고 문서를 적절한 크기로 분할한 후 임베딩 벡터를 생성하여 FAISS Vector DB 에 저장한다. 사용자가 정책과 관련된 질문을 입력하면 Retriever 가 가장 유사한 문서를 검색하고, 검색된 근거(Context)를 LLM 에 전달하여 출처 기반의 신뢰성 높은 답변을 생성한다. 정책 문서에 존재하지 않는 내용은 임의로 생성하지 않고 확인할 수 없음을 안내하도록 구현하여 Hallucination 을 최소화하였다.

프로젝트는 MCP(Model Context Protocol)를 적용하여 주문 조회, 재고 조회, FAQ 검색, 교환 신청 등의 기능을 표준화된 Tool 형태로 제공한다. MCP Server 는 외부 기능을 일관된 인터페이스로 노출하며, AI Agent 는 필요한 기능을 자동으로 발견하고 호출하여 결과를 활용할 수 있다. 이를 통해 시스템은 외부 서비스와의 연계성을 높이고 향후 새로운 도구를 손쉽게 추가할 수 있는 확장 가능한 구조를 제공한다.

사용자의 대화 이력은 Thread Memory(InMemorySaver)를 이용하여 스레드 단위로 관리된다. 이를 통해 이전 대화 내용을 기억하고 멀티턴(Multi-turn) 대화를 자연스럽게 이어갈 수 있으며, 동일한 사용자가 연속적으로 질문하는 경우에도 이전 문맥을 유지하여 일관성 있는 답변을 제공한다.

FastAPI 는 프로젝트의 전체 API 서버 역할을 수행하며, REST API 와 Swagger(OpenAPI)를 함께 제공하여 개발자와 사용자가 손쉽게 기능을 테스트할 수 있도록 구성하였다. 웹 UI 에서는 OpenAI와 Gemini 모델 선택, 질문 입력, 응답 결과 확인, 처리 시간 확인, LangGraph 실행 경로 확인 등의 기능을 제공하며, API 를 이용하는 외부 시스템과도 쉽게 연동할 수 있다.

프로젝트에서 사용하는 데이터는 주문 정보, 재고 정보, FAQ 데이터, 정책 PDF 문서 등으로 구성되어 있으며, CSV 데이터는 Tool 을 통해 직접 조회하고 정책 문서는 RAG 를 이용하여 검색한다. 이러한 구조는 정형 데이터와 비정형 문서를 동시에 활용할 수 있는 하이브리드 AI 시스템을 구현하는 데 목적이 있다.

전체 시스템은 FastAPI, LangChain, LangGraph, FAISS, OpenAI API, Gemini API, Pydantic, Uvicorn 등의 최신 기술을 활용하여 구축되었으며, ReAct 기반 추론, RAG 기반 문서 검색, A2A 기반 협업 에이전트, MCP 기반 Tool 호출을 하나의 통합 플랫폼으로 구현하였다.

이를 통해 다양한 업무 환경에서 정확하고 신뢰성 높은 AI 서비스를 제공할 수 있으며, 향후 데이터베이스, Vector DB, 외부 API, 다양한 전문 에이전트를 추가하여 더욱 확장 가능한 AI 플랫폼으로 발전시킬 수 있도록 설계하였다.

AI 에이전트 활용한 업무 자동화

업무자동화 개념: RPA 에서 에이전트 자동화로

- 목표를 기준으로 데이터를 수집·판단·실행·검증하는 루프를 에이전트가 수행하도록 설계
- RPA(규칙 기반) vs 에이전트 자동화(학습·판단 기반)

구분	RPA(규칙 기반)	에이전트 자동화(학습·판단 기반)
로직	고정 규칙, 화면/위치 의존	목표 기반 계획·툴 콜링
변경 내성	UI 변경에 취약	소스·경로 변경에 상대적 탄력
예외 처리	사람 개입 필수	자체 재시도·근거 제시·HITL 연계
통합	단일 앱 자동화 강점	다수 API/시스템 오케스트레이션 강점
구축/유지	빠른 도입, 유지비 증가	설계 난도 ↑, 확장성·재사용성 ↑
적합 업무	반복·규칙·고정 UI	데이터 탐색·요약·판단·보고

직무별 적용 사례(구매·조달 에이전트 파이프라인)

- ❖ **데이터→인사이트→협상→리포트**
 - 에이전트가 팀으로 움직이면 구매의 속도·정확도가 동시에 향상
- ❖ **에이전트 파이프라인**
 - Data Agent: 공급업체·시장지표 수집(LME/환율/CDS, 납기·결함률, 납품이력)
 - Compare Agent: 규격/단가/Incoterms/SL(서비스레벨) 정규화·비교표 생성
 - Negotiation Draft Agent: 계약조건 차이 분석 → 협상안 초안(BATNA/양보안·요청안)
 - Scenario Agent: 원자재·환율 $\pm x\%$ 변동 시뮬레이션(총소요비용, 민감도)
 - Report Agent: 의사결정 요약(3줄) + 표/차트 + 근거URL/타임스탬프

직무별 적용 사례(마케팅 에이전트 파이프라인)

- 고객 목소리(UGC)에서 인사이트를 뽑고, 크리에이티브를 만들고, A/B 실험으로 학습

❖ 에이전트 파이프라인

- Listen Agent - 리뷰/커뮤니티/콜로그 수집·정제(언어 감지, 중복 제거)
- Insight Agent - 주제 군집·감성/의도 분석 → 페인포인트·구매동인 도출
- Creative Agent - 페르소나·채널별 카피/이미지 브리프 생성(톤&매너 규칙 적용)
- Experiment Agent - 변형안 N개 생성 → A/B 테스트 설계(유의수준·표본수)
- Report Agent - 성과요약(CTR/CPA/전환) + 다음 실험 제안 + 근거 링크

직무별 적용 사례(재무 에이전트 파이프라인)

- 정형/비정형 데이터를 통합해 위험을 조기 탐지하고, 시나리오별 재무영향을 시뮬레이션

❖ 에이전트 파이프라인

- Data Ingest Agent - 회계시스템/ERP/시장지표 수집·정제(통화/기간 정렬)
- Accounting Rules Agent - 회계정책/세무 규정 룰 적용·분개 검증(예: IFRS 업데이트 반영)
- Risk & Forecast Agent - 유동성·신용·시장위험 지표 산출, 현금흐름/손익 예측
- Scenario Agent - 금리/환율/원자재 $\pm x\%$ 시나리오의 손익·B/S 영향 분석
- Report Agent - 월차/분기 보고서 초안, 경영진 1p 요약(핵심지표·경고신호·권고안)

직무별 적용 사례(인사조직 에이전트 파이프라인)

- AI는 HR의 '결정자'가 아니라 '균형자'(편향을 줄이고 데이터로 판단을 보조)

❖ 에이전트 파이프라인

- Profile Agent - 이력서/경력/성과 로그 수집·정규화(개인정보 비식별화)
- Match Agent - 직무역량 매핑·적합도 스코어 산출(스킬/경험/문화 적합도)
- Interview Agent - 역할·레벨별 질문 बैं크/평가 루브릭 자동 생성
- People Insight Agent - 설문/슬랙·메일 메타데이터로 조직 분위기·리스크 신호 탐지
- L&D Coach Agent - 개인·팀 단위 맞춤형 교육 로드맵 추천(업무 로그 기반)
- HR Report Agent - 채용/배치/평가 요약 및 편향감사 리포트(근거 링크 포함)

직무별 적용 사례(고객 서비스 에이전트 파이프라인)

■ 의도 파악 → 분류 → 해결/이관 → 학습(FAQ 업데이트) Closed Loop

❖ 에이전트 파이프라인

- Intent Agent - 문의 텍스트/음성에서 의도·감정 분석, 긴급도 산정
- Router Agent - 난이도·권한 기준 Tier 분류(Self-serve, Agent Assist, Human Expert)
- Solve Agent - 지식베이스+툴 호출로 답변/절차 실행(주문조회, 환불요청 등)
- Escalation Agent - 복잡/민감 케이스 전문가 이관 + 컨텍스트 요약 전달
- Learning Agent - 새 질의·해결 절차를 FAQ/지식베이스 자동 갱신
- QA Agent - 응답 품질 점검(정확도·톤·정책 준수) 및 피드백

오픈소스 LLM 구축 절차

단계	주요 내용	실무 관점 설명
요건 정의	비즈니스 목표, KPI, 데이터 경계, 보안 등급 설정	"AI를 왜 구축하는가?"를 명확히 해야 비용 낭비를 줄일 수 있음. 예시: '사내 문서 요약 자동화' vs '협력사 데이터 분석'
환경 세팅	GPU 서버, 클라우드, Docker, Conda 구성	실제로는 내부망 환경에서 설치 가능 여부가 핵심 포인트. (예시: 인터넷 제한, 방화벽 설정 등)
모델 선택	용도·성능·데이터 크기 고려	예시: '국문 QA 중심' → EXAONE / '다국어 리서치' → Mistral / '영문 기술문서 요약' → Llama 3
파인튜닝 (미세조정)	사내 데이터 특화 학습	IT팀만이 아니라 업무 전문가의 피드백 데이터가 필요. (예시: "이 표현은 우리 회사 용어가 아님")
배포 및 API 연결	FastAPI, Flask 기반 서버 구축	사내 시스템(SRM, ERP 등)과 연계 가능해야 함. 'Chat Interface' 형태로 적용 권장
모니터링 및 개선	성능, 비용, 보안 관리	비용은 GPU 사용률, 성능은 사용자 피드백, 보안은 로그·접근 이력 중심으로 점검

필요 기술 요소

구분	주요 기술	실무적 의미
ML/DL 프레임워크	PyTorch, TensorFlow	LLM 학습 및 실행의 핵심 엔진. 대부분의 오픈소스 모델이 PyTorch 기반.
벡터DB	FAISS, Weaviate, Pinecone	문서·텍스트를 숫자 벡터로 저장하여 검색 정확도를 높임 (RAG의 핵심 기술).
오케스트레이션	LangChain, LlamaIndex	여러 AI 모델과 데이터 소스를 연결해 "AI 워크플로우"를 자동화.
MLOps	MLflow, Kubeflow	모델의 버전·학습·배포를 통합 관리. 사내 LLM의 품질과 비용을 제어.
임베딩 모델	bge-m3, e5 등	텍스트 의미를 벡터로 변환해 검색과 문맥 연결 가능하게 함.
보안 및 관리	데이터 암호화, 접근 제어, 로그 대시보드	사내망 환경에서 안전하게 AI 모델을 운용하기 위한 필수 요소.

3. 확장 방향

여기까지 **LLM 호출 → 도구 → 루프 → RAG → 메모리 → 멀티에이전트 → 설계·배포 → 통합** 까지 에이전트 개발의 전 과정을 직접 구현했습니다. 이 골격을 **여러분의 실제 서비스**로 키우는 확장 방향과, 더 깊이 갈 다음 학습 주제를 정리합니다.

- 1: 두뇌 깨우기 — LLM 호출·프롬프트·추론
- 2: 손발 달기 — 도구·함수 호출·루프·계획·검색·코드실행
- 3: 지식과 기억 — RAG(문서 검색)·임베딩·벡터DB·메모리
- 4: 협업 — 워크플로우·멀티에이전트·역할 분담
- 5: 제품화 — 설계·운영·통합

빈 두뇌(LLM)에서 시작해 **진짜로 일하는 통합 CS 에이전트**까지 만들었습니다.

배포

- 29강 FastAPI를 Docker로 패키징 → 클라우드(AWS/GCP) 배포
- 메모리를 InMemorySaver → Redis/DB로 영속화 (서버 재시작에도 대화 유지)
- 환경별 config(dev/prod) 분리 운영

평가

- 테스트 질문 세트로 정확도 · 환각률 · 응답시간 측정
- LangSmith 등으로 추적 · 관측(observability)
- 회귀 테스트: 프롬프트 바꿨을 때 품질이 떨어지지 않는지

멀티에이전트 (도구가 늘면)

- 28강 판단대로, 도구가 5~7개를 넘으면 25~27강 패턴으로 분리
- 예: '결제팀 에이전트', '배송팀 에이전트' (Supervisor 라우팅)
- StateGraph로 복잡한 협업 흐름 선언(27강)

개인화 (장기 메모리)

- user_profiles.json 을 연결해 사용자별 정보 기억(20강)
- "지난번에 산 이어버드 어때요?" 같은 개인화 응대

안전장치

- 환불 실행 같은 비가역 행동에 human-in-the-loop 승인(07번)
- 입력 검증 · 가드레일(프롬프트 인젝션 방어, 3강)
- 비용 상한 · rate limit (29강)

더 깊이 갈 다음 학습 주제

다음 네 방향으로 심화할 수 있습니다.

- ① 운영 배포 · 관측성
 - LangSmith / OpenTelemetry 로 프로덕션 추적
 - 로그 · 메트릭 · 트레이스로 에이전트 동작 분석
- ② 평가 자동화
 - 골든 데이터셋, LLM-as-judge, A/B 테스트
 - "프롬프트를 바꿨더니 정말 좋아졌나"를 수치로
- ③ 멀티에이전트 고도화
 - LangGraph 심화(서브그래프, 휴먼 인 더 루프, 스트리밍)
 - 복잡한 협업 · 조건부 분기 · 병렬 실행
- ④ 안전성 · 가드레일
 - 프롬프트 인젝션 방어, 출력 검증, PII 마스킹
 - 책임 있는 AI(거부 · 면책 · 출처 표기)

실전 적용 — 여러분의 데이터로

가장 중요한 다음 단계는 여러분의 실제 데이터로 같은 골격을 적용하는 것입니다.

승승장구물 → 여러분의 회사/서비스

- orders.csv	→ 여러분의 주문/거래 DB
- 정책 PDF	→ 여러분의 사내 문서/매뉴얼
- faq.csv	→ 여러분의 FAQ/지식베이스

이 과정의 코드는 **데이터만 바꾸면** 여러분의 도메인에 그대로 적용됩니다. 도구의 CSV 경로를 바꾸고, RAG에 여러분의 문서를 넣고, 시스템 프롬프트를 여러분의 톤으로 다듬으면 됩니다. 골격은 그대로, 데이터만 여러분 것으로.

마지막 조언

- 작게 시작하라: 도구 1~2개로 동작하는 걸 먼저 만들고 늘려라
- 측정하라: "좋아진 것 같다"가 아니라 수치로 확인하라
- 필요한 것만: 멀티에이전트 · 장기메모리는 정말 필요할 때(28강 원칙)
- 운영을 잊지 마라: 모델이 좋아도 설정 · 로깅 · 재시도 없으면 제품이 아니다(29강)

가장 중요한 건 **계속 만들어 보는 것**입니다. 이 과정에서 만든 골격은 출발점이고, 진짜 실력은 여러분의 문제에 적용하며 쌓입니다.